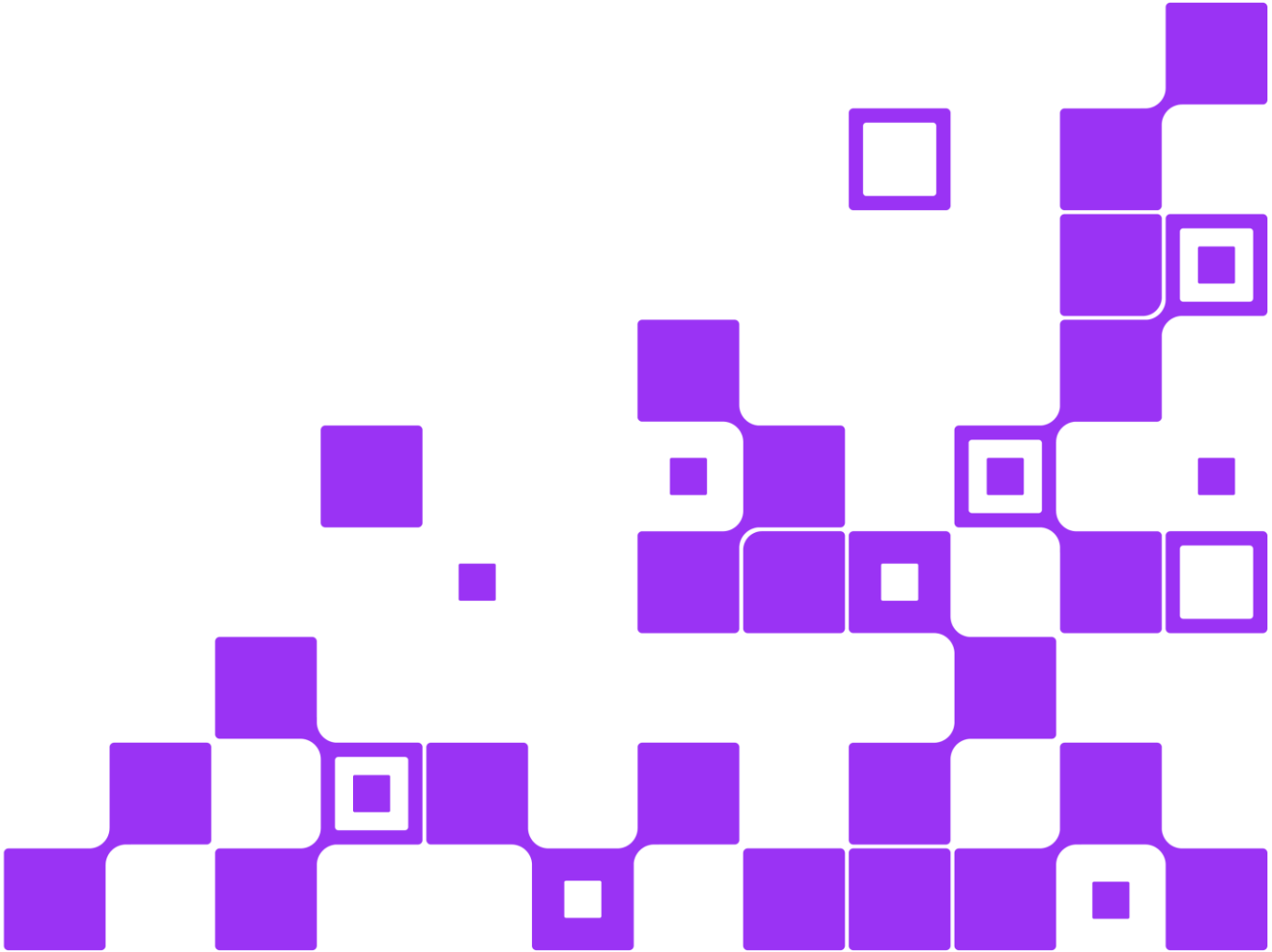
IThub

String



IThub

String — это класс Java, предназначенный для работы со строками, то есть с последовательностями символов.

- для хранения текста
- для работы с пользовательским вводом
- для чтения и обработки файлов
- для работы с JSON, XML и HTTP-запросами

В Java String является **классом** из пакета java.lang. Поэтому строковая переменная **хранит ссылку** на объект: `String text = "Hello";`

Одно из самых важных свойств String — **неизменяемость** (**immutability**).После создания объекта String его содержимое изменить нельзя.

Создание строк

IThub

Из массива символов

```
char[] chars = 'n', 'e', 'w', 'S', 't', 'r', 'i', 'n', 'g'};
String str = new String(chars);
String str2 = new String(chars, 3, 6);
```

Использовать строковый литерал

```
String str1 = "Hello";
```

Можно явно создать объект

```
String str3 = new String("Hello");
```

Длина строки - int length()

IThub

```
public void creationExample() {
    char[] chars = {'n', 'e', 'w', 'S', 't', 'r', 'i', 'n', 'g'};
    String str = new String(chars);
    System.out.println("str1 = " + str + " length = " + str.length());

    String str2 = new String(chars, 3, 6);
    System.out.println("\nstr2 = " + str2);

    String str3 = new String(str2);
    System.out.println("\nstr3 = " + str3);

    String str4 = "new String";
    System.out.println("\nstr4 = " + str4);
}
```



Конкатенация строк

IThub

Конкатенация — операция объединения строк, что возвращает новую строку, что есть результатом объединения второй строки с окончанием первой.

```
String text = "Hello";
text = text + " World";  // == text.concat(" World")
```



Создается новая строка: "Hello World“ и переменная text начинает ссылаться на новый объект.

String Pool

IThub

Для строковых литералов Java использует специальную область памяти, которая называется **String Pool**.

```
String a = "Hello";
String b = "Hello";
```



В этом случае JVM может использовать один и тот же объект для обеих переменных и оператор a == b может вернуть true (сравнивает ссылки)

String Pool

IThub

```
String a = "Hello";
String b = new String("Hello");
```



"Hello" находится в String Pool, а new String("Hello") создаёт новый объект String.
a == b вернет false

intern()

IThub

Метод intern() возвращает строку из пула строк, которая имеет такое же содержимое.

```
String s1 = new String("Java");
String s2 = “Java”

System.out.println(s1 == s2); // =false
```

Найди строку с таким содержимым в String Pool и верни ссылку на неё

```
String s1 = "Java";
String s2 = new String("Java");

String s3 = s2.intern();

System.out.println(s1 == s3); //=true
```

А если строки нет в пуле?

IThub

Если строки с таким содержимым ещё нет в String Pool, вызов intern() обеспечит её появление в пуле и вернёт ссылку на строку из пула.

IThub

1 Создаём строку через литерал

```
String string = "new String";
```

String Pool
(в области кучи)

"new String"
(одна копия)

Heap
(обычная куча)

(ничего не создается)

Строка "new String" уже существует в String Pool, поэтому создаётся только ссылка на неё.

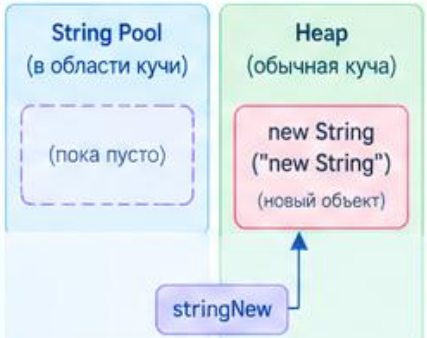
`string` → `"new String"` (из String Pool)

IThub

2 Создаём строку через new (когда ранее не создавали строку через литерал)

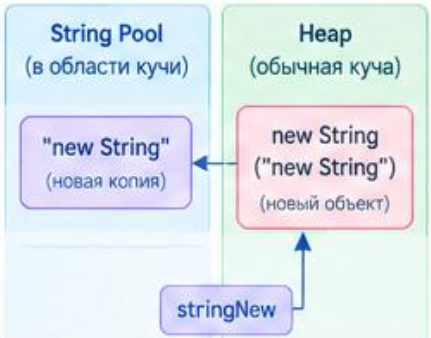
```
String stringNew = new String("new String");
```

2a Создаётся объект в Heap



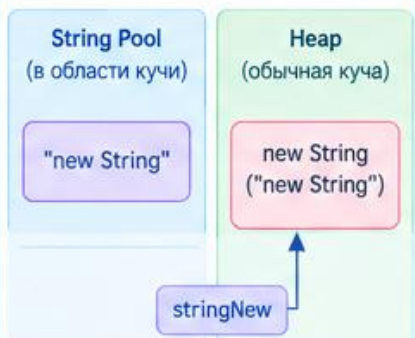
Выделяется новый объект String в Heap со значением "new String".

2b Создаётся копия в String Pool



Значение "new String" добавляется в String Pool (если его там ещё не было).

2c Переменная ссылается на объект в Heap

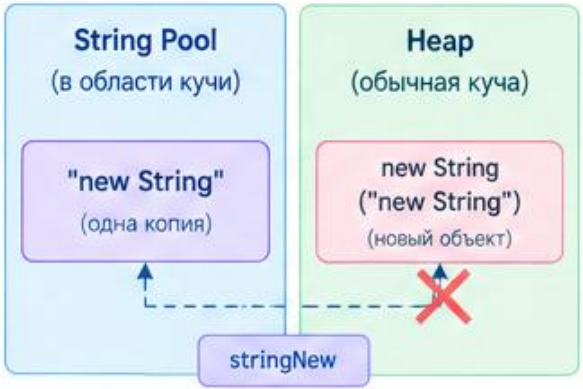


Переменнаа stringNew ссылается на объект в Heap, а не на строку из String Pool.

IThub

3 Метод intern()

```
stringNew.intern();
```



Метод intern() проверяет, есть ли строка в String Pool. Если есть — возвращает ссылку на неё. Если нет — добавляет строку в String Pool и возвращает на неё ссылку.

В результате stringNew будет ссылаться на объект из String Pool (а не на объект в Heap).

Сравнение строк

IThub

```
String a = "Hello";
String b = new String("Hello");
```



Для сравнения строк чаще всего используется метод **equals**.
Если необходимо сравнивать строки без учёта регистра, можно использовать метод **equalsIgnoreCase.**

```java
public void equalsExample() {
    String str1 = "Hello World";
    String str2 = "hello world";

    System.out.println("str1 = " + str1 + " str2 = " + str2);
    System.out.println(" equals = " + str1.equals(str2));

    System.out.println("\nstr1 = " + str1 + " str2 = " + str2);
    System.out.println(" equals ignore case = " +
str1.equalsIgnoreCase(str2));
}
```

```
str1 = Hello World str2 = hello world
 equals = false

str1 = Hello World str2 = hello world
 equals ignore case = true
```

Форматирование

IThub

Метод **format** позволяет вставлять значения переменных в строку в нужном формате.

```
String name = "Alex";
int age = 25;

String result = String.format("Меня зовут %s, мне %d лет.", name, age);

System.out.println(result);
```

Форматирование

IThub

Спецификатор	Значение	Пример
%s	строка	"Alex"
%d	целое число	25
%f	число с плавающей точкой	12.5
%b	boolean	true
%c	символ	'A'
%n	перенос строки	—

Форматирование

IThub

Спецификатор	Значение	Пример
%s	строка	"Alex"
%d	целое число	25
%f	число с плавающей точкой	12.5
%b	boolean	true
%c	символ	'A'
%n	перенос строки	—

Поиск в строках

IThub

- `int indexOf(int ch)` `int indexOf(String str)`

ищет первое вхождение символа или подстроки

- `int lastIndexOf(int ch)` `int lastIndexOf(String str)`

ищет последнее вхождение символа или подстроки

Возвращают индекс, по которому был найден символ или подстрока, либо -1, если символ или подстрока не обнаружены

Можно задать начальную позицию для поиска:

- `int indexOf(int ch, int startIndex)` `int indexOf(String str, int startIndex)`
- `int lastIndexOf(int ch, int startIndex)` `int lastIndexOf(String str, int startIndex)`

IThub

```java
public void searchExample() {
    String str = "May the Force be with you";

    int indexF = str.indexOf(70);
    System.out.println("index of F (70 on ASCII) = " + indexF);

    int indexForce = str.indexOf("Force");
    System.out.println("index of Force = " + indexForce);

    int lastIndexW = str.indexOf(119);
    System.out.println("lastIndex of w (119 on ASCII) = " + lastIndexW);

}
```

```
index of F (70 on ASCII) = 8
index of Force = 8
lastIndex of w (119 on ASCII) = 17
```

Модификация строк

IThub

- Извлечь подстроку

```
String substring(int startIndex)
String substring(int startIndex, int endIndex)
```

- Конкатенация двух строк

```
String concat(String str)
```

- Замена всех вхождений одного символа в вызывающей строке другим символом

```
String replace(char original, char replacement)
```

- Замена одной последовательности символов другой

```
String replace(CharSequence original, CharSequence replacement)
```

- Удаление ведущих и конечных пробелов в строке (пробелами считаются символы с кодами 32 и меньше)

```
String trim()
```

```java
public void modificationExample() {
    String str1 = "May the Force";
    String str2 = " be with you";

    String concat = str1.concat(str2);
    System.out.println("concatenation = " + concat);

    String subsrt = concat.substring(0, 3);
    System.out.println("Substring between o and 3 = " + subsrt);

    String replace = concat.replace("you", "us");
    System.out.println("replace = " + replace);

    String trim = str2.trim();
    System.out.println("str2 with trim = \"" + trim + "\"");
}
```

```
concatenation = May the Force be with you
Substring between o and 3 = May
replace = May the Force be with us
str2 with trim = "be with you"
```

Класс StringBuilder

IThub

Так как String неизменяем, большое количество операций конкатенации может приводить к созданию множества временных объектов.

Для частого изменения содержимого строки лучше использовать изменяемый класс StringBuilder.

StringBuilder позволяет изменять внутреннее содержимое объекта без создания новой строки при каждой операции.

Класс StringBuilder

IThub

- Добавить строку в конец объекта StringBuilder

```
StringBuilder append(String str)
```

- Вставить одну строку в другую

```
StringBuilder insert(int index, String str)
```

- Изменить на противоположный порядок следования символов

```
StringBuilder reverse()
```

- Удалить символы

```
StringBuilder delete(int startIndex, int endIndex)
```

- Заменить один набор символов другим

```
StringBuilder replace(int startIndex, int endIndex, String str)
```

```java
public void stringBuilderExample() {
    StringBuilder str1 = new StringBuilder("May the Force");
    StringBuilder str2 = new StringBuilder(" be with you");

    str1.append(str2);
    System.out.println("after append str1 = " + str1);

    str1.insert(25, "!");
    System.out.println("after insert str1 = " + str1);

    str1.reverse();
    System.out.println("after reverse = " + str1);

    str1.delete(0, 1);
    System.out.println("after delete str1 = " + str1);

    str1.reverse();
    str1.replace(22, 25, "us");
    System.out.println("after replace = " + str1);

    String substr = str1.substring(0, 3);
    System.out.println("substring between 0 and 3 = " + substr);
}
```

```
after append str1 = May the Force be with you
after insert str1 = May the Force be with you!
after reverse = !uoy htiw eb ecroF eht yaM
after delete str1 = uoy htiw eb ecroF eht yaM
after replace = May the Force be with us
substring between 0 and 3 = May
```

IThub

Три основных класса для работы со строковыми данными:

String
Неизменяемый

- удобен для хранения текста
- безопасен для использования в качестве значения
- широко используется в API

StringBuilder
Изменяемый

- эффективен при множественных изменениях строки
- используется в однопоточном коде

StringBuffer
Изменяемый

- методы синхронизированы
- используется в многопоточном коде

Null и пустая строка

IThub

Пустая строка:

String text = "";

Объект существует, но содержит 0 символов.

Проверка на null:
if (text == null) {
….

Проверка на пустую строку:
if (text.isEmpty()) {
…

isBlank() считает строку из пробелов пустой
if (text.isBlank()) {
…

null:

String text = null;

означает, что переменная не ссылается на объект String.

text.length() приведёт к NullPointerException.

Задача (КТ4)

Создайте консольное приложение на Java, которое позволяет управлять списком задач с использованием коллекций и файлов.

1. Создайте класс **Task**, который будет представлять базовую задачу. Включите в него поля: название, описание и статус (завершена или нет).
2. Создайте подкласс **PriorityTask**, который будет наследоваться от Task. Добавьте дополнительное поле - приоритет (например, "низкий", "средний", "высокий").
3. Создайте класс **TaskManager**, который будет содержать коллекцию задач. Используйте интерфейс List для хранения задач. Реализуйте методы для добавления и удаления задач из списка, а также для изменения статуса задачи.
4. Используйте классы FileOutputStream и FileInputStream для сохранения и загрузки списка задач из файла. Реализуйте методы для сохранения и загрузки задач в/из файла.
5. Реализуйте интерфейс Comparable для класса Task, чтобы задачи можно было сравнивать по приоритету.
6. Создайте класс ImmutableTask, который будет аналогичен классу Task, но будет иметь все поля с ключевым словом final. Покажите, какие могут возникнуть проблемы при попытке изменить поля. Объясните, что такие классы можно использовать в ситуациях, где требуется гарантированная неизменяемость.
7. В main методе, создайте объект TaskManager, добавьте несколько задач, выполните операции изменения статуса и сохраните список в файл. Затем загрузите список из файла и проверьте его корректность.